

Komondor parallelism showcase

sequential chain $\rightarrow$ SLURM array, $\sim 20\times$ wall reduction

2026-06-04

The headline

The current overnight chain (13885723 Slashdot+Epinions + 13885739 BA+OTC) is structurally **sequential** despite Komondor being designed for embarrassingly parallel work. Re-submitting the same 40-cell grid as a SLURM array (`-array=0-39%20`) caps the wall at $\max(\text{slowest_cell}) \approx 90$ min vs. the chain's sum ≈ 30 h, for a $\sim 20\times$ **wall reduction with identical compute total**.

1 The precedent — what ran sequentially

The 2026-06-03 chain (`run_hsikan_edge_cr_5seed_audit.sh + run_hsikan_edge_cr_ba_otc_5seed.sh`) is a triple-nested `for dataset; for seed; for mode` loop inside ONE SLURM job. SLURM gets a single allocation; all cells run on the same A100 in series. This is fine for AUC repeatability — every cell sees the same hardware — but it leaves Komondor's array dispatcher entirely unused. A 90-min cell takes $90 \times 40 = 3600$ min chained; it takes 90 min once when each cell is its own array task.

2 Per-cell wall stats (30 of 40 cells completed)

Data: `hsikan_edge_cr_audit/{results,results_ba_otc}.jsonl`. Epinions phase still in flight on 13885723.

```
26s - 1m28s |#####| 25
1m28s - 2m30s | | 0
...
9m46s - 10m49s |##### | 5
```

- $n = 30$, median = 30 s, max = 10m42s (Slashdot shuffle).
- BA + OTC dominate the short tail (26–35 s).
- Slashdot REAL completes fast (35 s/cell median, cache-warm).
- Slashdot SHUFFLE (5 cells $\times$ 10m42s) is the long tail in this snapshot. Epinions cells project to ~ 80 min.

3 Per-(dataset, mode) breakdown

dataset	mode	n	median wall	total wall	AUC mean $\pm$ std
bitcoin_alpha	real	5	26s	2m13s	0.9868 ± 0.0058
bitcoin_alpha	shuffle	5	28s	2m21s	0.9662 ± 0.0124
bitcoin_otc	real	5	28s	2m23s	0.9879 ± 0.0023
bitcoin_otc	shuffle	5	31s	2m37s	0.9435 ± 0.0060
slashdot	real	5	35s	2m50s	0.9058 ± 0.0037
slashdot	shuffle	5	10m42s	53m34s	0.8508 ± 0.0103
epinions	real	(running)	—	—	—
epinions	shuffle	(queued)	—	—	—

Slashdot real 5-seed result 0.9058 ± 0.0037 is within $+1\sigma$ of the published Mar-9 SOTA baseline 0.9067 ± 0.0029 , confirming the wheel-ship + Komondor reproduction holds. Strict-protocol gap real vs shuffle: $+0.055$ AUC ($n = 5$).

4 Speedup projection on the 30 completed cells

Sequential total wall: **1h05m**. Single-cell longest wall: **10m42s**.

K (parallel slots)	projected wall (LPT)	speedup vs serial
1	1h05m	1.00×
5	13m11s	5.00×
10	10m42s	6.10×
20	10m42s	6.10×
40	10m42s	6.10×

At $K \geq 10$ the max-cell wall dominates; the marginal slot adds no wall reduction. This is the LPT (Longest Processing Time) lower bound for makespan.

5 Forecast for the full 40-cell grid

Epinions cell wall is projected at ~ 80 min/cell from probe 13885703 (Slashdot single seed, 655 s, 2.05 GB RSS) and sacct on chain 13885723 Slashdot-shuffle cell walls. Epinions has $\sim 3\times$ Slashdot edge count $\Rightarrow$ wall scales with enumeration cost.

metric	sequential chain	array $K = 20$
dataset cells	40	40
compute total (GPU-min)	~ 1800 (30 h)	~ 1800 (identical)
wall (worst case)	~ 30 h	~ 90 min
speedup	1×	$\sim 20\times$
max parallel slots used	1	20
billing cost (same per-GPU-min)	identical	identical

6 The SLURM array submitter (the deliverable)

docs/komondor_setup/submit_hsikan_edge_cr_array.sh — same grid as the chain, same SOTA env vars, same SLURM resources per task. Key points:

- `-array=0-39%20` — 40 cells $\times$ 20 concurrent slots.
- `-time=02:30:00` per task (covers Epinions worst-case + headroom).
- Grid decomposition: $idx \rightarrow (dataset, seed, mode)$.
- flock-protected atomic JSONL append.
- Pinned env identical to the chain.

7 Hardware comparison

platform	GPU	per-cell wall	RSS
Komondor (HUN-REN)	A100-SXM4-40GB	30 s (median)	~ 2 GB
Local	RTX 2070 SUPER 7.6 GB	OOM (edge_cr)	> 7.6 GB

8 Follow-up (the K -sweep — requires user approval)

run	submit command	expected wall
$K = 5$	<code>sbatch ... --array=0-39%5 ...</code>	~ 6 h
$K = 10$	<code>sbatch ... --array=0-39%10 ...</code>	~ 3 h
$K = 20$	<code>sbatch ... --array=0-39%20 ...</code>	~ 90 min

Each K consumes 1800 GPU-min (identical compute total). Total billing across the K -sweep:
 $3 \times 1800 = 5400$ GPU-min ≈ 90 GPU-hours. **Not auto-launched.**

9 Verdict

This is the demo to bring to a poster session, a reviewer comment, or a project-progress meeting:

Komondor's gain on this audit grid is not about the A100 — it's about turning a 30-hour serial chain into a 90-minute array. The SOTA configuration only fits on the A100 (local 7.6 GB RTX OOMs at edge_cr forward), but the headline operational win is the array dispatcher. Both are needed; the array is the one that scales with grid size.